

Selling the service, not the source

A multi-tenant architecture for the assistant stack: how a client's domain reaches their own branded agent, where tenant identity is established, what has to change in the data layer before a second client can safely exist, and what it actually costs to serve a conversation.

Verified against the running stack, 2026-08-30 Companion to docs/SYSTEM-CHANGES.md

Status proposal

01 Problem and commercial goal

Clients get the service, never the source: they point a domain at our infrastructure, receive a branded assistant and a read-only analytics panel, and we keep control of branding, artifacts, and the code.

Topology is not what protects the source

Worth settling first, because it removes a false constraint from the decision. Whether we run one shared stack or one container per client, **the source code never leaves our infrastructure in either case**. The client receives HTTP responses. That is what protects the IP — not the container layout.

So the architecture gets chosen on **operating cost and maintainability**, not on intellectual-property grounds. Those two criteria point clearly in one direction.

THE FOUR QUESTIONS ANSWERED HERE

1. What is the right multi-tenant architecture for *this* stack?
2. How does a client's domain get connected, and what does onboarding look like?
3. Who pays for inference, how is it metered, and what do we charge?
4. Should registration be whitelist-only or open?

02 Where the stack is single-tenant today

Single-tenant everywhere except the browser. The only tenant-awareness that exists is cosmetic: one branding document and localStorage key namespacing.

STATE	LOCATION	CURRENT KEY OR SCOPE
Answer + audio cache	Redis	faq:<djb2 hash> — six nodes, no tenant prefix
Semantic cache vectors	Qdrant <code>faq_cache</code>	point id = <code>parseInt(questionHash, 16)</code>
Knowledge base	Qdrant <code>local_docs</code>	collection hardcoded, <code>options: {}</code> — no filter
Conversation memory	n8n process RAM	client-supplied <code>sessionId</code> , fallback <code>default-session</code>
Analytics	Postgres	no tenant column; 164 rows today
Admin passcode	Postgres <code>admin_settings</code>	one global row
Typo lexicon	Postgres <code>admin_settings</code>	one global row
Domain knowledge	AI Agent system message	inlined in the workflow
STT vocabulary	Groq STT node	inlined in the workflow
Timezone	<code>Asia/Beirut</code>	hardcoded in the overview SQL
Branding	<code>branding/branding.json</code>	one file per container
Brand assets	<code>ws-server.ts:161</code>	fixed global filenames
Hardware WS relay	<code>ws-server.ts:224</code>	two process-global sets, broadcasts to all
Credentials	n8n credential store	6 shared, ids hardcoded in nodes

The frontend is already tenant-ready: `TenantConfig.id` namespaces browser storage, and `scope.ts` applies it to chat sessions and the voice-retry IndexedDB. **The browser is done; nothing server-side has started.**

CAUTION — CHANGES HOW YOU READ ANY WORKFLOW CLAIM

GET `/api/v1/workflows/:id` returns **two different graphs**. Top-level `nodes` is the unpublished editor draft; `activeVersion.nodes` is the published version actually serving traffic.

```
running = w['activeVersion']['nodes']    # serves
traffic
draft   = w['nodes']                     # what the
editor shows
```

On 2026-08-30 these differed by **17 nodes**. Reading the top-level array produces a convincing false alarm that the semantic cache has been removed from production.

LIVE HAZARD — ACT ON THIS INDEPENDENTLY

That destructive draft is still sitting in the editor (`versionId 802bfc1d...` vs `activeVersionId 41bcb7c4...`). Anyone who opens Agent Workflow and clicks Publish ships it, and the Tier-2 semantic cache plus all three typo-tolerance layers vanish from production with no error. Restore the draft from the active version, or discard it.

03 The decision: pooled, not container-per-tenant

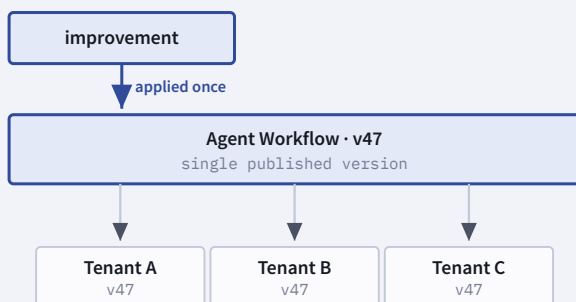
RECOMMENDATION

One shared stack with `tenant_id` threaded through every layer, plus a dedicated-stack Enterprise tier that reuses the same image and the same `tenant_id` code path.

Why not a stack per tenant

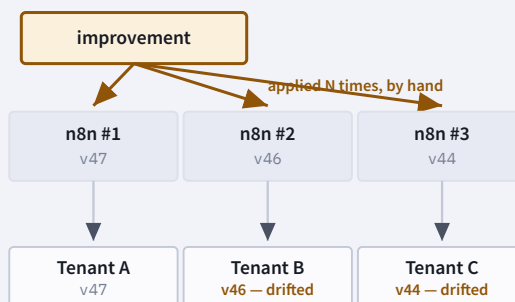
The decisive argument is not RAM — it is **workflow maintenance**. The Agent Workflow is 47 nodes carrying the two-tier cache, the LLM judge, the typo lexicon and the TTS chunking, and it is under continuous improvement. Fork that per tenant and every future improvement becomes N manual edits across N n8n instances, applied by hand through a UI, with no diff and no rollout control. The forks drift, and the drift stays invisible until a tenant reports a bug that exists only in their copy.

Pooled one graph, all tenants



`tenant_id` separates the data;
the graph stays identical.

Stack per tenant N copies of the graph



Drift is invisible until a tenant reports
a bug that exists only in their copy.

The difference is what happens on the next improvement. Pooled, it ships once and every tenant has it. Siloed, it must be re-applied by hand to each instance, and any instance that gets missed silently diverges — which is the failure mode that removes the ability to ship at all.

Secondary arguments: n8n idle RSS is roughly 700 MB–1 GB, so a handful of tenants exhausts a normal VPS; N Postgres instances means N backup and migration regimes; and `docker-compose.yml` hardcodes `container_name` on every service and uses host bind mounts throughout, so it cannot be run twice on one host without being rewritten anyway.

Why pooled fits this stack specifically

The per-tenant surface is small and almost entirely *data*: a system prompt, a document corpus, a lexicon, a branding document, a timezone, a set of colours. None of it is code. The workflow graph is identical for every tenant.

The Enterprise tier keeps the option open

A client who contractually requires physical isolation gets a dedicated stack — the same image, the same `tenant_id` code path, with exactly one tenant in it. Because isolation lives in the data model rather than the deployment, a silo is a *configuration* of the pooled design, not a fork of it.

	POOLED (DEFAULT)	ENTERPRISE SILO
Stack	shared	dedicated
Workflow updates	once, everywhere	once per silo
Data isolation	logical (<code>tenant_id</code>)	physical
Bring-your-own API keys	not possible	yes
Cost to serve	marginal	a full stack

04 Request lifecycle

Where tenant identity is established is the most important detail in the design.



Every store below the control plane is keyed by a tenant id the client never supplied.

Tenant identity is derived once, server-side, from the validated Host — and never accepted from anything the client controls. Caddy strips the header a caller might try to forge; the control plane resolves the tenant and injects config downstream, so n8n does no extra I/O on the cache-hit path.

05 Domains and TLS

Two ways in, deliberately

- `<tenant>.ourplatform.com` — provisioned instantly, covered by one wildcard certificate via DNS-01. Available the moment a tenant is created, which matters for sales demos.
- A **customer domain** (`amba.client.com`) — the client creates a CNAME to our ingress; the certificate is issued automatically on first request.

Subdomain-only would weaken the white-label story the branding work exists to serve. Custom-only means nothing works until DNS propagates, so there is no instant demo URL.

On-demand TLS and the ask endpoint

```
{
  on_demand_tls {
    ask http://control-plane:3001/internal/tls-check
  }
}
```

Caddy sends `GET /internal/tls-check?domain=amba.client.com` at handshake time for an unknown SNI. **Any 2xx issues the certificate; anything else refuses.** The response body is ignored, and once a cert is cached there are no further asks until renewal.

THREE RULES, ALL LOAD-BEARING

1. **Fail closed.** If the database is unreachable, return 5xx. A handler that returns 200 on error lets anyone pointing DNS at our IP mint certificates until we hit the Let's Encrypt rate limit and are locked out for a week.
2. **Exact match only** against `tenant_domains`. No wildcards, no suffix matching.
3. **Rate-limit the handler itself.**

Restricting on-demand TLS to customer CNAMEs — platform subdomains ride the wildcard — keeps the blast radius to the CNAME set and keeps subdomain provisioning instant.

Tenant resolution happens in the app, not in Caddy

CORRECTS THE OBVIOUS FIRST DESIGN

Caddy cannot look up a tenant id in a database. `{host}` gives the hostname, and a `map` directive gives only a static translation table that needs a config reload on every new tenant — which defeats the purpose.

So Caddy does what it is good at: TLS termination, certificate gating, header hygiene.

```
reverse_proxy control-plane:3001 {  
  header_up -X-Tenant-Id  
  header_up X-Forwarded-Host {host}  
}
```

The control plane holds the Host→tenant map in memory and resolves in nanoseconds. One less moving part, and one less place for the mapping to go stale.

06 The control plane

`web/ws-server.ts` is already a real HTTP server handling `/api/*` . Promote it into its own service and give it the jobs that currently have no home.

JOB	TODAY	AFTER
Serving the app	Vite dev server in production	the built bundle from <code>web/dist</code>
Tenant resolution	does not exist	Host → tenant, in memory
Auth	shared passcode, two mechanisms	users table, argon2, cookie sessions
Branding	one JSON file	per-tenant record, resolved by host
Analytics	<code>/webhook/admin-stats</code> in n8n	SQL directly against Postgres
Webhook access	Vite proxy	authenticated proxy to internal-only n8n

Why kill Vite in production

`web/vite.config.ts` runs the **dev server** with `host: "0.0.0.0"` and no `allowedHosts` , started by `bun run dev` in `web/Dockerfile:15` . A dev server is not designed to face the internet — the Vite 5.4.x line has had arbitrary-file-read CVEs — and putting one behind a proxy that accepts arbitrary per-tenant Host headers is not a posture worth defending.

The replacement is easy. `bun run build` already exists and `web/dist` is already produced. Routing is **hash-based** (`main.tsx:29` matches `#/admin`), so the only real path the browser requests is `/` — no SPA-fallback complexity. The three Vite proxy rules become a few lines of `Bun.serve` URL rewriting, and Bun streams bodies, so multipart audio upload and base64 audio responses work unchanged.

Auth model

Replace all three of today's mechanisms — the Postgres passcode hash, the plaintext `ADMIN_PASSCODE` env var, and the passcode held in `sessionStorage` — with one.

- `users` table: `tenant_id` , email, argon2id hash, role.
- Roles: `operator` (us, global), `tenant_admin` , `tenant_viewer` .
- Cookie sessions: HTTP-only, Secure, SameSite=Lax, scoped to the tenant's domain.
- Entitlements as **per-tenant capability flags** — `can_edit_branding` , `can_edit_lexicon` — rather than hardcoded restrictions.

MIGRATION ORDERING

The passcode is the only working auth in the system. Keep it functioning in parallel until the session path has been exercised, then remove it in a separate change. Do not do both at once.

The two panels

	CLIENT PANEL	OPERATOR CONSOLE
Where	their own domain, <code>#/admin</code>	<code>console.ourplatform.com</code>
Analytics	their tenant only	any tenant
Usage / spend	their tenant only	all tenants, plus cost
Question log + CSV	yes	yes
Branding, theme, avatar	no	yes
Documents, prompt, lexicon	no	yes
Tenant CRUD, domains, invites	no	yes
View-as-tenant	no	yes

CHEAPER THAN IT LOOKS

The existing `web/src/components/admin/` tree needs **no changes**. `lib/adminApi.ts` is already a clean `AdminClient` interface with a swappable implementation. If the new endpoints return the existing typed shapes byte-identically, swapping `/webhook/admin-stats` for `/api/admin/*` is a **single-file change**. The Branding tab moves to the operator console essentially as-is.

07 **Data-layer isolation**

Two of these are not cosmetic namespacing — they are the difference between a mixup and a cross-tenant data leak.

Postgres — genuinely cheap

All three analytics queries funnel through a **single leading CTE** holding the only reference to the base table. Verified against the published workflow:

QUERY	SIZE	BASE-TABLE REFS	CHOKE POINT
Overview Query	6,741 chars · 17 CTEs	1	scoped
Log Query	1,170 chars	1	filtered
Corrections Query	1,222 chars	1	scoped

So the tenant predicate is **one line per query, three lines total**:

```
WITH scoped AS (  
  SELECT * FROM receptionist_session_logs  
  WHERE timestamp >= $1::timestampz AND timestamp <= $2::timestampz  
         AND tenant_id = $3                -- the entire change  
) ,
```

```
ALTER TABLE receptionist_session_logs ADD COLUMN tenant_id TEXT;  
UPDATE receptionist_session_logs SET tenant_id = 'iu1' WHERE tenant_id IS NULL;  
ALTER TABLE receptionist_session_logs ALTER COLUMN tenant_id SET NOT NULL;  
ALTER TABLE receptionist_session_logs ADD COLUMN execution_id TEXT; -- metering  
  
-- every index gains tenant_id as the leading column  
DROP INDEX idx_rsl_timestamp;  
CREATE INDEX idx_rsl_tenant_ts  
  ON receptionist_session_logs (tenant_id, timestamp DESC);  
  
ALTER TABLE admin_settings ADD COLUMN tenant_id TEXT NOT NULL DEFAULT 'iu1';  
ALTER TABLE admin_settings DROP CONSTRAINT admin_settings_pkey;  
ALTER TABLE admin_settings ADD PRIMARY KEY (tenant_id, key);
```

'iu1' is the correct backfill value — it is the id in branding.json already flowing into setTenantScope(). Only 164 rows exist, so the backfill is instant.

TWO ORDERING TRAPS

`postgres_init/init.sql:45` seeds the lexicon with `ON CONFLICT (key) DO NOTHING`, and the live `Lexicon Save` node does `ON CONFLICT (key) DO UPDATE`. Both must become `ON CONFLICT (tenant_id, key)` or they break the moment the primary key changes.

Redis — a prefix, six places

`t:<tenant>:faq:<hash>`. Six nodes build these keys today, all as expressions like `={{ 'faq:' + $json.questionHash }}`. Mechanically simple; just do all six.

Qdrant — two real problems

PROBLEM 1 — A MANDATORY PAYLOAD FILTER FAILS OPEN

The intuitive design (one shared collection, a `tenant_id` payload field, a filter on every search) has the wrong failure mode. If `tenantId` ever resolves empty, the filter vanishes and the retriever returns **every tenant's documents** into the agent context, which is then read aloud to a stranger. No error, no alert, and the log row looks completely normal.

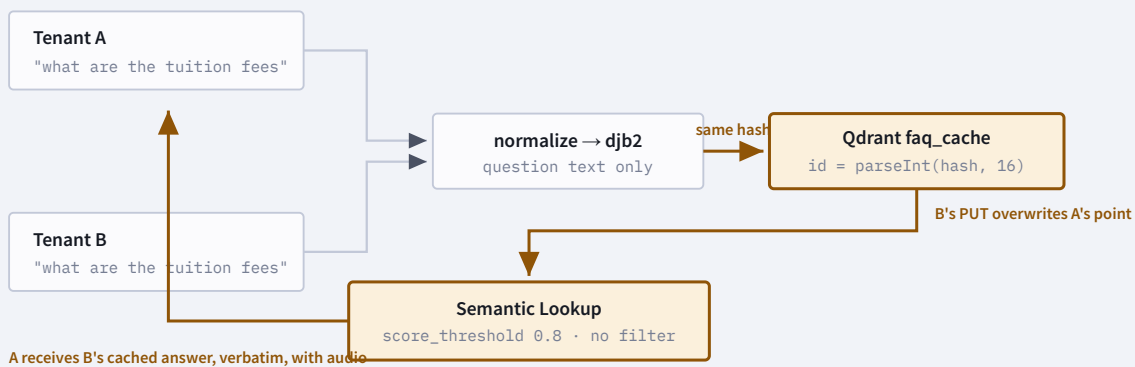
Use per-tenant collections instead (`docs_<tenant>`). The same failure produces a nonexistent collection name and a hard 404 — it *fails closed*.

This is cheap: the Qdrant node's `qdrantCollection` is a resource locator in `mode: "id"`, which accepts an expression. Revisit the shared-collection plus `is_tenant` scheme only past a few hundred collections — and budget it properly then, because `is_tenant` needs a keyword payload index and per-tenant HNSW subgraphs need `hnsw_config: { m: 0 }`, which cannot be set in place. That is a collection recreate plus a full re-embed at 3072 dimensions.

PROBLEM 2 — THE SEMANTIC CACHE WILL SERVE ONE TENANT'S ANSWER TO ANOTHER

And a Redis prefix does not fix it. From the published `Index Question Vector` node:

```
const id = parseInt(nh.questionHash, 16);
url:
'http://qdrant:6333/collections/faq_cache/points',
body: { points: [{ id, vector: vec, payload: {
  questionHash: nh.questionHash, question:
correctedQuestion } }] },
```



The point id is derived from the question text alone. Two tenants asking the same thing collide on one Qdrant point; the judge then correctly answers SAME, because the questions genuinely are identical. The fix is a UUIDv5 over `tenant_id + questionHash`, or per-tenant cache collections — point ids must be an unsigned integer or a UUID, so a string prefix is not available.

ONE PRINCIPLE NEEDS SPLITTING

Both Qdrant nodes swallow all errors by design — "indexing is best-effort, never break the response over it." That is right for availability and **wrong for an isolation control**. A tenancy filter that silently no-ops is worse than a hard failure. Isolation failures should throw; everything else keeps degrading gracefully.

Conversation memory

`sessionKey` is currently `={{ ...body?.sessionId || 'default-session' }}` — client-supplied, unauthenticated, with a shared fallback bucket that pools every anonymous turn across all tenants.

- Prefix it: `t:<tenant>:chat:<sessionId>` .
- **Mint the session id server-side** (signed, tenant-bound) rather than trusting a browser value from `Math.random()` .
- Make a missing `sessionId` a hard 400 rather than a shared bucket.

AND MOVE OFF MEMORYBUFFERWINDOW ENTIRELY

It is in-process RAM. The moment n8n runs queue mode with multiple workers, consecutive turns from the same user land on different workers and the buffer is empty half the time — conversation memory silently degrades to none, with no error. Prefixing the key does not fix this. Use Redis chat memory; Redis is already in the stack.

08 n8n as a shared execution plane

THE LARGEST PIECE OF PREREQUISITE WORK

One n8n instance in its current configuration is **not safe to share between tenants**. This is cheaper to fix before there are paying tenants than after.

Four compounding problems

- **SQLite.** No `DB_TYPE` is set, so n8n runs on SQLite. The database is **240 MB with an 11.5 MB WAL** from a single tenant's pilot traffic. SQLite is single-writer and every execution writes synchronously — this arrives as latency on the user path, not a clean error.
- **No pruning.** There is no `EXECUTIONS_DATA_PRUNE*` anywhere (verified: zero matches). That is why the file is 240 MB after a pilot.
- **Single process, unbounded concurrency.** One Node event loop, no `N8N_CONCURRENCY_PRODUCTION_LIMIT`. And `Build JSON (Text + Audio)` does `Buffer.concat` over raw PCM and hand-builds a WAV header in-process, for every audio turn, for every tenant. **One tenant's demo day is every tenant's outage.**
- **No isolation primitive.** Every credential is global, and there is no `networks: block`, so any Code node can reach `http://qdrant:6333` unauthenticated — as `Semantic Lookup` already does.

PREREQUISITES BEFORE TENANT #2

```
DB_TYPE=postgresdb           # off SQLite
EXECUTIONS_MODE=queue         # + Redis, 2+ workers
N8N_CONCURRENCY_PRODUCTION_LIMIT=... # bound the blast radius
EXECUTIONS_DATA_PRUNE=true
EXECUTIONS_DATA_MAX_AGE=168   # must exceed the metering drain window
```

Queue mode also provides the natural lever for tier isolation: a dedicated worker pool per Enterprise tenant, same image, same code path.

Inject tenant config; do not query for it

Trace the graph: `Webhook` → `Switch` → ... → `Normalize & Hash Question` → `Check Redis Cache`. **The Redis lookup is nearly the first thing that happens**, and it needs the tenant id to build its key. So tenant config must resolve before the fastest path in the system — the cache hit measured at 290–340 ms.

Loading it with a Postgres query inside the workflow adds a synchronous round-trip to every turn, including cache hits, on one event loop. That taxes the cheapest, highest-volume path to serve the rarest one. Instead, **the control plane merges resolved tenant config into the request body before forwarding.**

THE SECURITY COROLLARY IS NOT OPTIONAL

The webhook has **no authentication at all** today. Once tenant config arrives in the request body, anyone who can reach `n8n:5678` can impersonate any tenant, override the system prompt, and point the vector store at another tenant's collection. n8n must be on an internal-only network, and the webhook node needs a Header Auth credential holding a secret only the control plane knows.

What is expression-driven

Confirmed against the published workflow — each already uses expressions elsewhere, so the pattern is proven in production.

PER-TENANT FIELD	NODE	MECHANISM
System prompt	AI Agent	<code>options.systemMessage</code> — plain string
Knowledge collection	Qdrant Vector Store	resource locator, <code>mode: "id"</code>
STT vocabulary	Groq STT	multipart body param <code>prompt</code>
Memory key	Window Buffer Memory	<code>sessionKey</code> — already an expression
Cache keys	six Redis nodes	already expressions
Cache collection	two raw-HTTP Code nodes	our own request bodies

THE CURLY-BRACE FOOTGUN

The Tools Agent composes a LangChain prompt template from the system message using f-string interpolation. A tenant prompt containing `{ }` or `{ }` — a JSON example, or `Hi {name}` — produces *Missing value for input variable* and the agent **hard-fails on every turn**. Reject or escape braces at write time, and run a smoke test against a tenant's prompt before it goes live.

Blast radius

With one workflow for all tenants, a bad edit takes down every tenant simultaneously. There is no canary and no per-tenant pin. Minimum mitigations: server-side prompt validation, a synthetic turn before publish,

and a workflow-version pin per tenant so a change can be rolled to a subset first. The `activeVersion` mechanism from §2 is what this would build on.

09 Metering and what a turn costs

VERIFIED — EXECUTION 743, A FULL FRESH TURN

```
Gemini (Agent LLM): {promptTokens: 1200,
completionTokens: 25}
Gemini (Agent LLM): {promptTokens: 0,
completionTokens: 20}
Gemini (Tool LLM): {promptTokens: 201,
completionTokens: 62}
Judge Same Question: {promptTokenCount: 135,
candidatesTokenCount: 3}
Gemini TTS: {promptTokenCount: 40,
candidatesTokenCount: 380,
candidatesTokensDetails:
[{"modality": "AUDIO", ...}]}
```

Three things this settles:

- The LangChain Agent node **hides token usage from its output** (downstream nodes see only `{output: string}`), but n8n's tracing callback records it in **execution data**. It is retrievable.
- **One turn is 4–5 LLM calls, not one.** The agent loop re-sends the growing context each step. Billing a turn as one call under-bills by roughly 4–5×.
- The TTS node reports `modality: "AUDIO"` separately — exactly what is needed to price it at audio rates.

How to read it

1. Add `execution_id` to the log table, set from `$execution.id` in `Anonymize Log Entry` (that node already runs on every path).
2. A control-plane worker drains rows where `metered_at` IS NULL, calls the executions API, sums `tokenUsage` across every `ai_languageModel` sub-run plus `usageMetadata` from the HTTP nodes, and writes one `usage_events` row per turn.
3. Belt-and-braces: capture `usageMetadata` from the three HTTP nodes synchronously into the log row too. TTS is the dominant cost and is **already fully observable** without any of the above.

TWO CONSTRAINTS

This creates a hard dependency on execution retention — the worker must drain faster than the prune window, and its failure must be loud.

And **do not estimate tokens from character counts**. Traffic is bilingual; Gemini's tokenizer runs roughly 1 token per 1–2 characters for Arabic versus ~4 for English, so one estimator misprices tenants by around 3× depending on language mix — and it cannot see retrieved documents or the accumulating tool-loop context, which is where most of the tokens are.

What a turn actually costs

MEASURED COUNTS, INDICATIVE RATES — VERIFY BEFORE PUBLISHING ANY PRICE

COMPONENT	MEASURED	INDICATIVE RATE	COST
Text input	1,576 tok	\$0.30 / 1M	\$0.00047
Text output	110 tok	\$2.50 / 1M	\$0.00028
TTS audio output	380 tok	~\$10 / 1M	\$0.0038
STT (voice turns)	~10 s	~\$0.04 / hr	\$0.0001
Embedding	1 call	—	negligible
Fresh voice turn			≈ \$0.0046
Cache hit	Redis GET only		≈ \$0

TTS is ~82% of the marginal cost of a turn. And a cache hit is essentially free, because Redis stores the audio too — which is why cache entries average ~0.55 MB.

THE FINDING THAT SHOULD SHAPE PRICING

At a 60% cache-hit rate the blended cost is about **\$0.0018 per conversation**. A tenant doing 5,000 conversations a month costs roughly **\$9 in inference** — far below the infrastructure they sit on. **Marginal cost per tenant is dominated by fixed infrastructure, not by usage.**

1. **Do not build metering to recover inference cost** — there is barely any to recover. Build it for *abuse protection* (a runaway loop or a scraper, not normal usage) and for *tier definition*.

2. **Flat-rate pricing with generous limits is correct.** Usage-based billing would add billing complexity, procurement friction, and customer anxiety to recover single-digit dollars.
3. **The cache is a margin engine, not just a latency feature.** A cache hit costs nothing and earns full price. The two-tier cache and typo-tolerance work raise gross margin directly — a much better argument for that work than "it feels faster."

STILL A PILOT

With 164 log rows, all volume figures are projections. Re-derive them from `usage_events` once real traffic exists.

The bring-your-own-key answer

PER-TENANT API KEYS CANNOT WORK ON THE POOLED TIER

n8n binds credentials at design time and offers **no expression-driven credential selection**. The Gemini nodes are pinned to credential `3NeEadTXR21D1oX6`, and the same is true of Qdrant, Postgres, Redis and Groq. There is no supported way for one shared workflow to select a tenant's own key at runtime.

This turns out well commercially:

- **Pooled tiers run on our keys.** Given the cost analysis, that is the right default anyway — the client is buying an assistant, not tokens, and \$9 of inference is not worth pushing onto their procurement process.
- **BYO keys become an Enterprise-tier feature.** That tier has its own stack and therefore its own credentials, and it is the tier whose buyers actually ask for it.

One path would unlock BYO keys on the pooled tier: replace the LangChain Agent node with a hand-rolled tool loop of HTTP Request nodes, where the key is an ordinary expression. That would also yield synchronous token counts and remove the execution-data dependency — but it means owning the agent loop. A costed future project, not a quick change.

The client's usage panel

Read-only, beside analytics, showing **conversations** — not tokens:

- conversations used vs. included this period
- cache-hit rate, framed as "answered instantly"

- projected overage, if any

For a BYO-key Enterprise tenant, the same panel shows **estimated provider cost** computed from our own token counts at published list rates, clearly labelled an estimate. We cannot read their Google billing account, and pretending otherwise would be wrong.

10

Commercial model

Bill conversations, not tokens. Track tokens internally for cost truth; expose conversations to the customer.

A turn is 4–5 agent calls plus an embedding, a judge, STT and TTS — tokens are not comparable across those, raw-token pricing exposes us to provider price changes, and "your plan includes 5,000 conversations a month" is far easier to sell than a token meter.

INDICATIVE PACKAGING — A STARTING POINT FOR A PRICING CONVERSATION, NOT NUMBERS TO PUBLISH

	STARTER	GROWTH	ENTERPRISE
Conversations / month		2,000	10,000 negotiated
Domain	platform subdomain	+ custom domain	+ custom domain
Documents	limited corpus	larger corpus	unlimited
Isolation	pooled	pooled	dedicated stack
BYO API keys	—	—	yes
Support	email	priority	SLA

Gross margin on the pooled tiers is very high, and the binding constraint is **fixed infrastructure per pooled cluster**, not per-tenant usage. Price on delivered value — a branded, document-grounded, voice-capable assistant with an analytics dashboard — rather than cost-plus.

11 Registration and onboarding

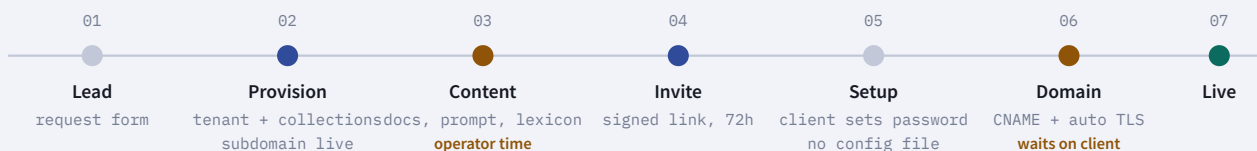
RECOMMENDATION – WHITELIST, NOT OPEN SELF-SERVE

Operator-provisioned, invite-only, with a public "request access" form for lead capture. The public site should capture demand, not provision it: a form that creates a *lead*, not a tenant.

Four reasons, all specific to this product:

1. **Every tenant needs a DNS change on their side.** That is already a human step self-serve cannot complete.
2. **Every tenant needs a document corpus ingested and a system prompt written.** That is service delivery, not a signup form — and today it is also a workflow edit.
3. **Every turn has real marginal cost.** Open signup without a card means paying for strangers' traffic, and there is no quota system yet.
4. **There is no billing system,** and B2B at dozens of accounts does not need self-serve — that is for thousands of small accounts.

Onboarding flow



The "configure it at startup" idea lands as step 05 – the invite wizard.
The client never touches a config file, and never needs an API key.

Steps 03 and 06 are the ones that actually consume time — content curation on our side, DNS propagation on theirs. Worth measuring on the first two tenants before promising an onboarding SLA.

Why the analytics-only boundary should still be a flag

The client panel is analytics-only by decision. Implement that as a per-tenant capability flag (`can_edit_branding` , default off) rather than a hardcoded restriction — identical behaviour today, and flipping it for one client later costs nothing.

Worth flagging as a tradeoff to revisit: withholding branding edits keeps visual consistency and creates a service touchpoint, but every tagline tweak becomes a support ticket. The flag makes that a per-account decision rather than an architectural one.

12

Migration path

FIX NOW – INDEPENDENT OF MULTI-TENANCY

ISSUE	EVIDENCE
Branding writes are unauthenticated in production	<code>docker-compose.yml:102</code> interpolates <code>\${ADMIN_PASSCODE}</code> but <code>.env:13</code> defines <code>ADMIN_DASHBOARD_PASSCODE</code> . Confirmed empty in the running container; the guard is <code>if (ADMIN_PASSCODE && ...)</code> , so an empty value disables the check .
The destructive workflow draft	One Publish click from removing the semantic cache and all typo tolerance.
No execution pruning	240 MB SQLite + 11.5 MB WAL.
<code>ws-server.ts</code> is not type-checked	<code>web/tsconfig.json:21</code> includes only <code>["src", "vite.config.ts"]</code> . It is about to become a production API service.
<code>CLAUDE.md</code> and <code>README.md</code> are stale	Both describe Ollama, Piper and Whisper as live. Inference is Gemini + Groq; those containers are vestigial or bypassed.

PHASE 1 – MUST LAND BEFORE TENANT #2

THE FIRST THREE ARE CORRECTNESS, NOT SCALING

1. n8n onto Postgres, queue mode, pruning, concurrency limit.
2. Qdrant per-tenant collections + UUIDv5 cache point ids — closes the cross-tenant answer leak.
3. Redis key prefixes (six nodes); Redis-backed chat memory with a server-minted, tenant-scoped session id.
4. Postgres migration: `tenant_id` , `execution_id` , index rewrites, composite `admin_settings` key, the two `ON CONFLICT` fixes.
5. Control plane split out: built bundle, Host→tenant resolution, config injection.
6. n8n on an internal-only network; Header Auth on the webhook.
7. Per-tenant config out of the workflow, with brace validation.

PHASE 2 – BEFORE SELLING

8. Caddy, wildcard cert, on-demand TLS with a fail-closed ask handler.
9. Real auth: users, argon2, cookie sessions, roles, capability flags.
Passcode retired *after* the new path is exercised.
10. Analytics moved off `/webhook/admin-stats` into the control plane.
11. Operator console: tenant CRUD, domains, invites, branding editor, view-as-tenant.
12. Metering worker and `usage_events` ; per-tenant rate limits and quotas.
13. Client usage panel.

PHASE 3 – AS DEMAND APPEARS

14. Enterprise silo tier (same image, one tenant), with BYO keys.
15. Tenant-scoped WS rooms and authenticated hardware clients; same-origin WS URL.
16. Per-tenant document upload, if the analytics-only boundary is ever relaxed.

ON ITEM 10

The tracked `admin_dashboard_workflow.json` has 13 nodes; the live workflow has 24. The export is missing the entire lexicon path (`Validate Lexicon` , `Lexicon Valid?` , `Lexicon Save`) and the corrections path — and `Lexicon Save` is a *write* with validation logic behind it. Reimplementing from the repo file would silently drop features. Work from a fresh `activeVersion` export.

13

Open questions and risks

#	QUESTION OR RISK	WHY IT MATTERS
1	Do we own a platform domain yet, and where is DNS hosted?	The wildcard cert via DNS-01 needs API access to the DNS provider. Blocks \$5.
2	Realistic tenant count in 12 months — 5, 50, or 500?	Under ~10, a simpler design might suffice. Past a few hundred Qdrant collections, per-tenant collections need revisiting.
3	Who operates this at 3am?	Pooled means one incident affects everyone. Queue mode and pruning reduce but do not remove that.
4	Is the per-tenant corpus curated by us indefinitely?	It is the main recurring operator cost, and the main argument for eventually giving clients document upload.
5	Verify provider rates before publishing any price.	Every cost figure in \$9 is indicative; the token counts are real, the rates are not verified.
6	Data residency and retention commitments?	The log table stores questions and answers. Multi-tenant means a per-tenant retention policy and an export/delete path. Nothing exists today.
7	Which tenant is the pilot for phase 1?	Migrating <code>iu1</code> in place is lower risk than onboarding a new client onto an untested path.

The three risks most likely to bite

- The n8n shared execution plane.** The largest, least visible piece of work. Doing it after there are paying tenants is far more expensive than doing it now.
- The Qdrant fail-open trap.** A tenancy control that silently returns another tenant's documents is the worst failure mode in the design. Per-tenant collections are the mitigation, and the "best-effort, never throw" principle must not be applied to isolation checks.
- Auth migration.** The passcode is the only working auth in the system. Replacing it means a window where new, untested code guards the operator console. Run both in parallel, then remove.